

Post-Quantum AI-Powered API Gateway

Complete Installation & Local Setup Guide

This guide describes the reproducible local installation procedure for the current capstone implementation using WSL2 Ubuntu. The system contains a C++ Crow gateway, liboqs post-quantum cryptography, a Python security engine, ZeroMQ communication, a Flask backend, and a browser dashboard.

1. Prerequisites

Recommended environment:

- Windows 11 with WSL2
- Ubuntu distribution under WSL2
- Internet connection for initial package/source installation
- A terminal with sudo access
- Git repository available at `~/post-quantum-api-gateway`

2. Open WSL2 Ubuntu

```
wsl
```

Verify the environment:

```
uname -a
lsb_release -a
which g++
which cmake
python3 --version
git --version
```

The project was developed and tested in WSL2 Ubuntu. Keep the development commands inside the Linux environment rather than mixing Windows and WSL paths.

3. Update Ubuntu

```
sudo apt update
sudo apt upgrade -y
```

4. Install System Dependencies

```
sudo apt install -y \
build-essential \
git \
cmake \
ninja-build \
pkg-config \
curl \
wget \
unzip \
openssl \
libssl-dev \
python3 \
python3-pip \
python3-venv \
python3-dev \
libzmq3-dev \
cppzmq-dev \
libasio-dev \
libcurl4-openssl-dev
```

These packages provide the compiler, build tools, Python environment, ZeroMQ development libraries, Asio/Crow dependencies, libcurl, and OpenSSL support.

5. Clone / Enter the Project

```
cd ~
git clone <YOUR-GITHUB-REPOSITORY-URL> post-quantum-api-gateway
cd ~/post-quantum-api-gateway
```

If the repository is already present, do not clone it again. Simply enter it:

```
cd ~/post-quantum-api-gateway
```

6. Verify Project Structure

```
find . -maxdepth 2 -type f | sort
```

The important directories are:

```
ai/
backend/
dashboard/
docs/
gateway/
tests/
```

7. Install / Build liboqs

The project uses liboqs for ML-KEM-768 and ML-DSA-65. Install it locally under ~/.local.

```
cd ~
git clone --depth 1 https://github.com/open-quantum-safe/liboqs.git
cd liboqs
mkdir build
cd build

cmake -GNinja \
  -DCMAKE_INSTALL_PREFIX=$HOME/.local \
  -DBUILD_SHARED_LIBS=ON ..

ninja
ninja install
```

Verify the installation:

```
ls ~/.local/lib
ls ~/.local/include/oqs
```

If liboqs was already installed during development, this step can be skipped after verifying the installation.

8. Configure the Python Virtual Environment

```
cd ~/post-quantum-api-gateway

python3 -m venv .venv
source .venv/bin/activate

python3 -m pip install --upgrade pip
pip install flask pyzmq
```

If the project later adds additional Python dependencies, install them inside this virtual environment.

9. Verify Crow / C++ Dependencies

The gateway uses Crow and Asio. The required Asio development package is installed with:

```
sudo apt install -y libasio-dev
```

libcurl support is installed with:

```
sudo apt install -y libcurl4-openssl-dev
```

10. Build the Gateway

```
cd ~/post-quantum-api-gateway

rm -rf gateway/build

cmake -S gateway -B gateway/build -GNinja
cmake --build gateway/build
```

A successful build should produce:

```
gateway/build/gateway
```

11. Verify the PQC Component

The project should be tested for ML-KEM-768 and ML-DSA-65 before starting the complete system. The expected cryptographic validation is:

```
ML-KEM-768 encapsulation : PASS
Gateway decapsulation    : PASS
Shared secret match      : PASS
ML-DSA-65 signing        : PASS
ML-DSA-65 verification   : PASS
```

12. Start the Backend

Open Terminal 1:

```
cd ~/post-quantum-api-gateway
source .venv/bin/activate
python3 backend/app.py
```

The backend listens on:

```
http://127.0.0.1:5000
```

13. Start the AI Security Engine

Open Terminal 2:

```
cd ~/post-quantum-api-gateway
source .venv/bin/activate
python3 ai/security_engine.py
```

The AI engine uses ZeroMQ. The gateway sends analysis events to port 5555 and receives security decisions on port 5556.

14. Start the C++ Gateway

Open Terminal 3:

```
cd ~/post-quantum-api-gateway
./gateway/build/gateway
```

The gateway listens on:

```
http://localhost:8080
```

15. Start the Dashboard

Open Terminal 4:

```
cd ~/post-quantum-api-gateway
python3 -m http.server 3000 --directory dashboard --bind 0.0.0.0
```

Open the dashboard in a Windows browser:

```
http://localhost:3000
```

16. Verify Gateway Health

```
curl -i http://localhost:8080/health
```

A successful response should indicate that the gateway and its security components are running.

17. Test Backend Proxying

```
curl -i http://localhost:8080/api/hello
```

Expected result:

```
HTTP/1.1 200 OK
```

18. Test SQL Injection Pre-Filter

```
curl -i -X POST http://localhost:8080/api/data \
-H "Content-Type: application/json" \
-d "{\"query\": \"' OR 1=1 --\"}"
```

Expected result:

```
HTTP/1.1 403 Forbidden
Request blocked by gateway pre-filter: SQL_INJECTION
```

19. Test Rate Limiting

```
for i in {1..30}; do
  curl -s -o /dev/null -w "%{http_code}\n" \
    http://localhost:8080/api/hello
done
```

The output should contain a mixture of 200 and 429 responses when the request burst exceeds the token-bucket allowance.

20. Check Metrics

```
curl -s http://localhost:8080/metrics
```

The metrics endpoint exposes traffic, AI, payload, and enforcement information used by the dashboard.

21. Check Security Events

```
curl -s http://localhost:8080/events
```

The event endpoint returns the latest in-memory security events, including timestamp, client IP, endpoint, event type, action, and attack type.

22. Reset Security State

```
curl -s -X POST http://localhost:8080/metrics/reset
```

The reset operation clears metrics, rate-limiter state, AI enforcement state, and the event log.

23. Optional Gemini Configuration

The Gemini/LLM component is optional. The local deterministic security engine works without an external LLM. Do not place API credentials directly into source code or commit them to Git. Use the project's ignored secret/environment mechanism if the optional hosted analysis is enabled.

24. Troubleshooting

Gateway does not build: verify CMake, Ninja, liboqs, Asio, libcurl, OpenSSL, and the compiler installation.

Gateway cannot find liboqs: verify ~/.local/lib and ~/.local/include/oqs and ensure the CMake configuration can locate the local installation.

Port 5000 is busy: stop the old Flask process before starting another backend instance.

Port 5555/5556 is busy: stop previous AI/gateway processes before restarting.

Dashboard shows no data: make sure the gateway is running on port 8080 and that the browser can access localhost:8080.

AI is disabled: this is acceptable for the deterministic local mode; the gateway still performs rate limiting and payload security.

25. Recommended Startup Order

- 1. Backend
- 2. Python AI Security Engine
- 3. C++ Gateway
- 4. Dashboard

Use four separate WSL terminals. This makes the inter-process flow easy to observe during demonstrations.

26. Final Verification Checklist

Check	Expected
Backend	Listening on 127.0.0.1:5000
AI engine	ZeroMQ analysis service running
Gateway	Listening on localhost:8080
Dashboard	Available on localhost:3000
ML-KEM-768	Encapsulation/decapsulation PASS
ML-DSA-65	Signing/verification PASS
Normal API	HTTP 200
Malicious pre-filter	HTTP 403
Rate limit	HTTP 429 under burst
Metrics	/metrics returns JSON
Events	/events returns JSON
Reset	Security state clears

This installation guide reflects the current capstone prototype. Production deployment would require additional work such as production TLS, persistent storage, distributed state, OS firewall integration, and high-availability deployment.